EXPERIMENT – 4

AIM :

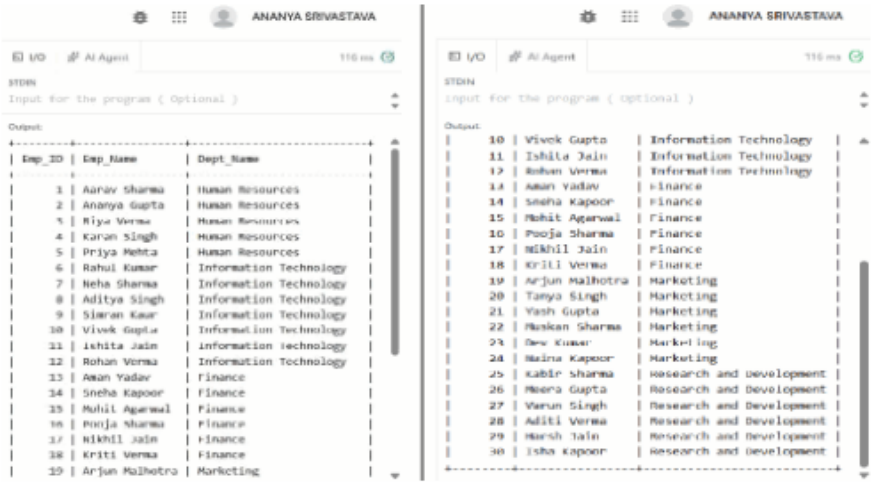
To write and execute SQL queries using the Employee schema demonstrating INNER JOIN, LEFT JOIN, SELF JOIN, 3-WAY JOIN, Correlated Subqueries, EXISTS, simulated INTERSECT and EXCEPT operations, and to compare query execution plans using EXPLAIN.

SOURCE CODE :

QUERY 1 – INNER JOIN

```
SELECT
   e.Emp_ID,
   e.Emp_Name,
   d.Dept_Name
FROM Employee e
INNER JOIN Department d
ON e.Dept_ID = d.Dept_ID;
```

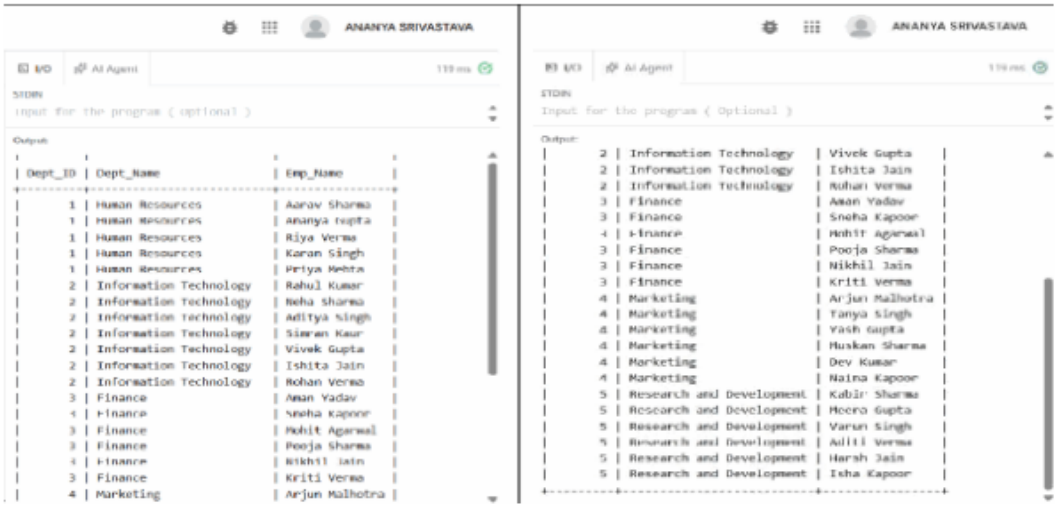
OUTPUT:



QUERY 2 – LEFT JOIN

```
SELECT
   d.Dept_ID,
   d.Dept_Name,
   e.Emp_Name
FROM Department d
LEFT JOIN Employee e
ON d.Dept_ID = e.Dept_ID;
```

OUTPUT:



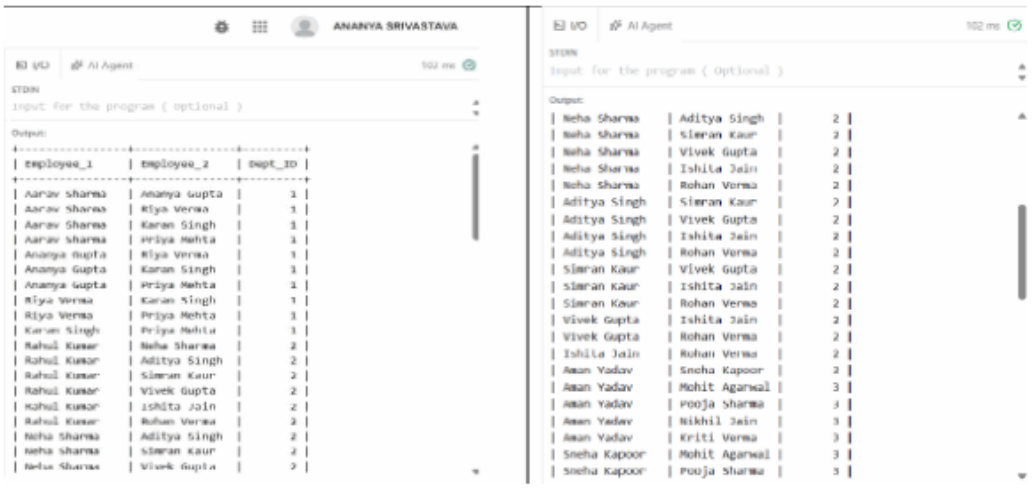
Dept_ID	Dept_Name	Emp_Name
1	Human Resources	Aarav Sharma
1	Human Resources	Ananya Gupta
1	Human Resources	Riya Verma
1	Human Resources	Karan Singh
1	Human Resources	Priya Mehta
2	Information Technology	Rahul Kumar
2	Information Technology	Neha Sharma
2	Information Technology	Aditya Singh
2	Information Technology	Simran Kaur
2	Information Technology	Vivek Gupta
2	Information Technology	Ishita Jain
2	Information Technology	Rohan Verma
3	Finance	Aman Yadav
4	Finance	Sneha Kapoor
3	Finance	Mohit Agarwal
3	Finance	Pooja Sharma
4	Finance	Nikhil Jain
3	Finance	Kriti Verma
4	Marketing	Arjun Malhotra

Dept_ID	Dept_Name	Emp_Name
2	Information Technology	Vivek Gupta
2	Information Technology	Ishita Jain
2	Information Technology	Rohan Verma
3	Finance	Aman Yadav
3	Finance	Sneha Kapoor
4	Finance	Mohit Agarwal
3	Finance	Pooja Sharma
3	Finance	Nikhil Jain
3	Finance	Kriti Verma
4	Marketing	Arjun Malhotra
4	Marketing	Tanya Singh
4	Marketing	Yash Gupta
4	Marketing	Muskan Sharma
4	Marketing	Dev Kumar
4	Marketing	Naina Kapoor
5	Research and Development	Kabir Sharma
5	Research and Development	Meera Gupta
5	Research and Development	Varun Singh
5	Research and Development	Aditi Verma
5	Research and Development	Harsh Jain
5	Research and Development	Isha Kapoor

QUERY 3 – SELF JOIN

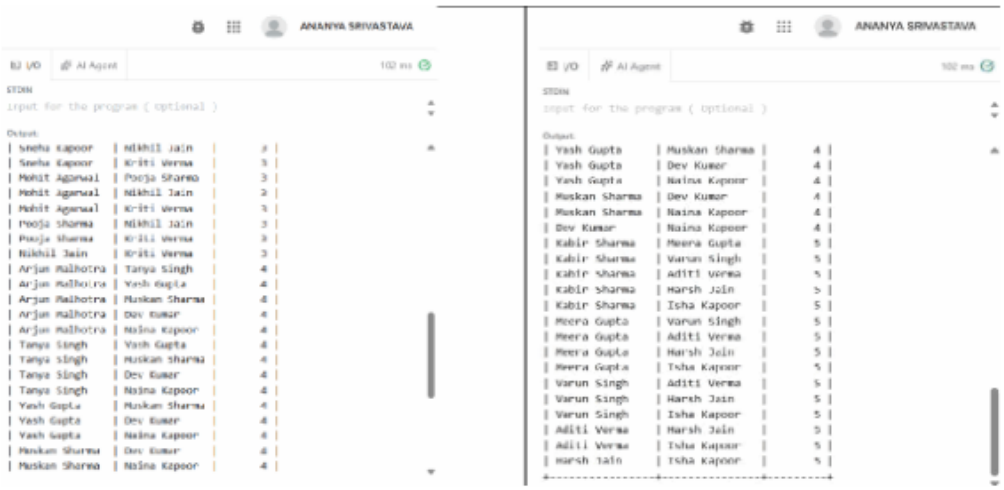
```
SELECT
   e1.Emp_Name AS Employee_1,
   e2.Emp_Name AS Employee_2,
   e1.Dept_ID
FROM Employee e1
INNER JOIN Employee e2
ON e1.Dept_ID = e2.Dept_ID
AND e1.Emp_ID < e2.Emp_ID;
```

OUTPUT:



Employee_1	Employee_2	Dept_ID
Aarav Sharma	Ananya Gupta	1
Aarav Sharma	Riya Verma	1
Aarav Sharma	Karan Singh	1
Aarav Sharma	Priya Mehta	1
Ananya Gupta	Riya Verma	1
Ananya Gupta	Karan Singh	1
Ananya Gupta	Priya Mehta	1
Riya Verma	Karan Singh	1
Riya Verma	Priya Mehta	1
Karan Singh	Priya Mehta	1
Rahul Kumar	Neha Sharma	2
Rahul Kumar	Aditya Singh	2
Rahul Kumar	Simran Kaur	2
Rahul Kumar	Vivek Gupta	2
Rahul Kumar	Ishita Jain	2
Rahul Kumar	Rohan Verma	2
Neha Sharma	Aditya Singh	2
Neha Sharma	Simran Kaur	2
Neha Sharma	Vivek Gupta	2

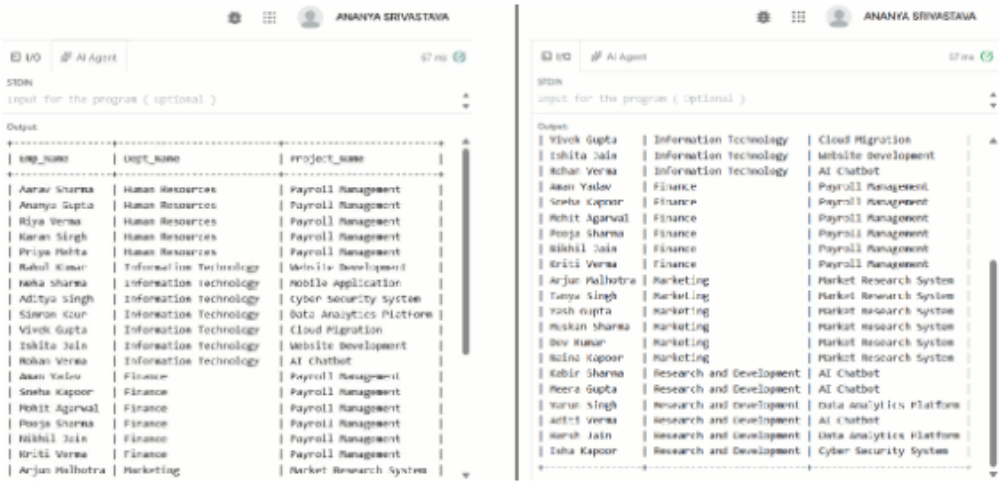
Neha Sharma	Aditya Singh	2
Neha Sharma	Simran Kaur	2
Neha Sharma	Vivek Gupta	2
Neha Sharma	Ishita Jain	2
Neha Sharma	Rohan Verma	2
Aditya Singh	Simran Kaur	2
Aditya Singh	Vivek Gupta	2
Aditya Singh	Ishita Jain	2
Aditya Singh	Rohan Verma	2
Simran Kaur	Vivek Gupta	2
Simran Kaur	Ishita Jain	2
Simran Kaur	Rohan Verma	2
Vivek Gupta	Ishita Jain	2
Vivek Gupta	Rohan Verma	2
Ishita Jain	Rohan Verma	2
Aman Yadav	Sneha Kapoor	3
Aman Yadav	Mohit Agarwal	3
Aman Yadav	Pooja Sharma	3
Aman Yadav	Nikhil Jain	3
Aman Yadav	Kriti Verma	3
Sneha Kapoor	Mohit Agarwal	3
Sneha Kapoor	Pooja Sharma	3



QUERY 4 – 3-WAY JOIN

```
SELECT
   e.Emp_Name,
   d.Dept_Name,
   p.Project_Name
FROM Employee e
INNER JOIN Department d
ON e.Dept_ID = d.Dept_ID
INNER JOIN Project p
ON e.Project_ID = p.Project_ID;
```

OUTPUT:



QUERY 5 – CORRELATED SUBQUERY

```
SELECT
   e.Emp_ID,
   e.Emp_Name,
   e.Salary,
   e.Dept_ID
FROM Employee e
WHERE e.Salary >
(
   SELECT AVG(e2.Salary)
   FROM Employee e2
```

```
    WHERE e2.Dept_ID = e.Dept_ID
);
```

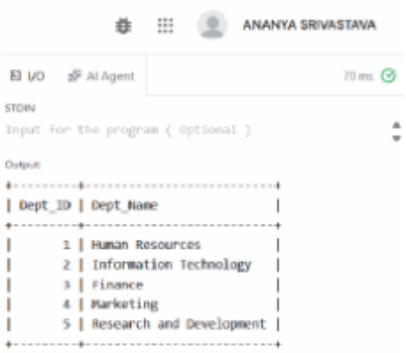
OUTPUT:



QUERY 6 – EXISTS

```
SELECT
    d.Dept_ID,
    d.Dept_Name
FROM Department d
WHERE EXISTS
(
    SELECT 1
    FROM Employee e
    WHERE e.Dept_ID = d.Dept_ID
);
```

OUTPUT:



QUERY 7 – SIMULATED INTERSECT USING EXISTS

```
SELECT
    e.Emp_ID,
    e.Emp_Name
FROM Employee e
WHERE e.Salary > 80000
AND EXISTS
(
    SELECT 1
```

```
    FROM Project p
    WHERE p.Project_ID = e.Project_ID
    AND p.Budget > 800000
);
```

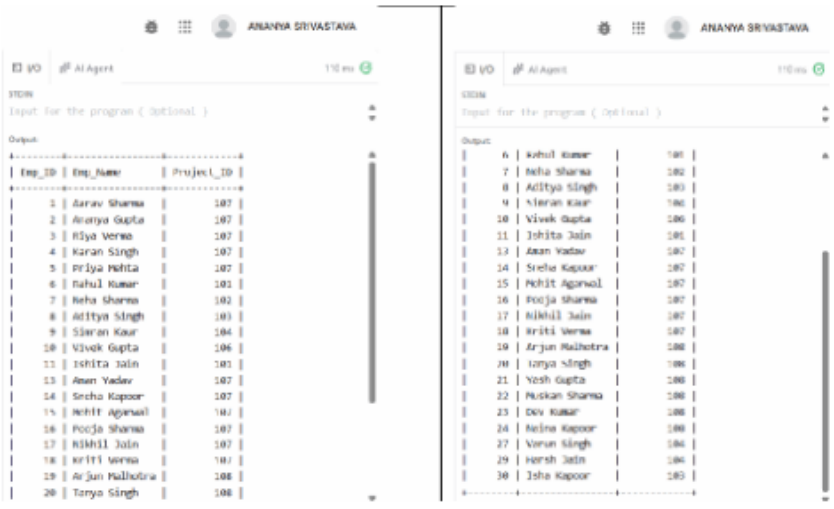
OUTPUT:



QUERY 8 – SIMULATED EXCEPT USING NOT EXISTS

```
SELECT
    e.Emp_ID,
    e.Emp_Name,
    e.Project_ID
FROM Employee e
WHERE NOT EXISTS
(
    SELECT 1
    FROM Project p
    WHERE p.Project_ID = e.Project_ID
    AND p.Project_Name = 'AI Chatbot'
);
```

OUTPUT:



QUERY 9 – EXPLAIN FOR INNER JOIN

```
EXPLAIN
```

```
SELECT
   e.Emp_ID,
   e.Emp_Name,
   d.Dept_Name
FROM Employee e
INNER JOIN Department d
ON e.Dept_ID = d.Dept_ID;
```

OUTPUT:



QUERY 10 – EXPLAIN FOR LEFT JOIN

```
EXPLAIN
SELECT
   d.Dept_Name,
   e.Emp_Name
FROM Department d
LEFT JOIN Employee e
ON d.Dept_ID = e.Dept_ID;
```

OUTPUT:



QUERY 11 – EXPLAIN FOR CORRELATED SUBQUERY

```
EXPLAIN
SELECT
   e.Emp_ID,
   e.Emp_Name,
   e.Salary
FROM Employee e
WHERE e.Salary >
(
   SELECT AVG(e2.Salary)
   FROM Employee e2
   WHERE e2.Dept_ID = e.Dept_ID
);
```

OUTPUT:



QUERY 12 – EXPLAIN FOR 3-WAY JOIN

```
EXPLAIN
SELECT
   e.Emp_Name,
   d.Dept_Name,
   p.Project_Name
FROM Employee e
INNER JOIN Department d
ON e.Dept_ID = d.Dept_ID
INNER JOIN Project p
ON e.Project_ID = p.Project_ID;
```

OUTPUT:



QUERY 13 – COMPARISON OF EXECUTION PLANS

INNER JOIN:

```
EXPLAIN
SELECT
   e.Emp_ID,
   e.Emp_Name,
   d.Dept_Name
FROM Employee e
INNER JOIN Department d
ON e.Dept_ID = d.Dept_ID;
```

Correlated Subquery:

```
EXPLAIN
SELECT
   e.Emp_ID,
   e.Emp_Name,
   e.Salary
FROM Employee e
WHERE e.Salary >
(
   SELECT AVG(e2.Salary)
   FROM Employee e2
```

```
   WHERE e2.Dept_ID = e.Dept_ID
);
```

COMPARISON:

The EXPLAIN output can be compared using the following attributes:

- Table – shows the table accessed by the query.
- Type – shows the type of table access or join operation.
- Possible_Keys – shows indexes that MySQL could potentially use.
- Key – shows the index actually selected by MySQL, if any.
- Rows – shows the estimated number of rows examined.
- Extra – provides additional information about query processing.

OUTPUT :

